

Training Compute-Optimal Large Language Models

Hoffmann et al., DeepMind — NeurIPS 2022

情報経営システム工学部

LOPEZ CHAPA JOSE

2026/06/09

Outline

What we will cover today

- **Act 1 — The Problem:** Why does this question matter? What did the field believe?
- **Act 2 — The Methodology:** Three independent approaches to find the optimal frontier
- **Act 3 — The Payoff:** Chinchilla, its results, and what it means for the field

ACT 1: The Problem

Why does the training allocation question matter?

The Reality of Training Large Models

Training is expensive, and you only get one shot

- The largest models in 2022 had over **500 billion parameters**
- Training requires **thousands of accelerators** running for **months**
- Compute and energy cost **risers with every increase in model size**
- The training compute budget is **known in advance** — fixed by hardware and time
- It is typically only feasible to train these models **once**

💰 Accurately estimating the best hyperparameters before committing the budget is critical.

So You Have a Budget. Now What?

The fundamental tension: model size vs. training data

You have a fixed compute budget C . There are exactly two places to spend it:

- ↑ **Model size** — more parameters, more capacity, richer representations
- ↑ **Training data** — more tokens, more learning opportunities, broader language exposure

You cannot maximize both simultaneously.

What Are We Choosing Between?

Parameters N

- The **weights** of the model — adjusted during training
- More N = more capacity to represent complex patterns
- Cost: more memory, more inference compute, more fine-tuning cost
- Think of N as the model's **knowledge capacity**

Training Tokens D

- Token $\approx$ **word or word fragment**
- D = total tokens processed during all of training
- Each token = one **learning opportunity**: predict $\rightarrow$ get corrected $\rightarrow$ update weights
- More D = broader exposure to language
- Think of D as the model's **accumulated experience**

Every extra parameter costs compute on every forward and backward pass. Every extra token costs compute on every training step. They compete directly under a fixed budget.

The FLOPs Equation

$$C \approx 6 \times N \times D$$

Symbol	Meaning	Notes
C	Total FLOPs consumed during training	Fixed by your hardware budget
N	Number of model parameters	The size of the model
D	Number of training tokens	The length of the training run
6	Operations per parameter per token	~2 forward + ~2 backward + ~2 weight update

Because N and D **multiply** to give C , and C is fixed:

$$\uparrow N \implies \downarrow D \quad \text{and} \quad \uparrow D \implies \downarrow N$$

Doubling model size halves the tokens you can afford. They are locked in direct tradeoff.

What Did the Field Actually Do?

Table 1: parameters grew, tokens stagnated

Model	Parameters	Training Tokens
LaMDA	137 Billion	168 Billion
GPT-3	175 Billion	300 Billion
Jurassic	178 Billion	300 Billion
Gopher	280 Billion	300 Billion
MT-NLG	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Why Everyone Thought Bigger Was Better

Kaplan et al. (2020) showed a **predictable power-law** between parameter count and loss — more parameters, lower loss, across many orders of magnitude. Their conclusion: **model size should grow faster than training data** with compute.

Error 1: Fixed learning-rate schedule

Kaplan used one cosine schedule designed around 130B tokens. Shorter runs were evaluated while the learning rate had not decayed properly, making short training horizons look worse than they should.

Error 2: Narrow model size range

Many runs were below 100M parameters. At larger scales, the FLOP-loss frontier shows curvature that is not visible in small-model-only data. Extrapolating the small-scale trend leads to the wrong optimum.

What This Paper Does

Experiment small → predict large → train once

- **Step 1:** Train over **400 models** from 70M to 16B parameters, on 5B to 400B tokens, each with a learning rate schedule matched to its actual training length
- **Step 2:** Use **three independent methods** to extract the optimal frontier — if all three agree, the result is robust
- **Step 3:** Train **Chinchilla** at the predicted optimal allocation — verify it outperforms models trained under the old paradigm

ACT 2: The Methodology

Three approaches to find the compute-optimal frontier

Three Approaches: A Map

	Approach 1	Approach 2	Approach 3
Strategy	Vary token horizons per model size	Fix FLOP budget, vary model size	Fit parametric loss function
Data used	Points along full training curves	Final loss of fully trained models	All losses from 1 & 2
Key output	Minimum loss envelope	Loss valley per budget	Closed-form efficient frontier
Result a	0.50	0.49	0.46
Result b	0.50	0.51	0.54

Three methods, different assumptions, different data — all converge on a similar proportion.

Approach 1: Scanning Across Training Lengths

Question: For each compute budget, which model + training length combo wins?

- Train many model sizes (70M → 10B), each for **4 different training lengths**
- For every FLOP budget, pick the run with **lowest loss**
- Fit a power law to the winners

The key insight: by varying training length independently of model size, you can directly observe where the budget is best spent.

$$N_{\text{opt}} \propto C^{0.50} \qquad D_{\text{opt}} \propto C^{0.50}$$

 [Figure 2: optimal N and D vs FLOPs]

Approach 2: IsoFLOP Profiles

Question: For a fixed budget, what is the best model size?

- Fix the compute budget, train many models of **different sizes**, all spending exactly that budget
- Plot final loss vs model size — a **clear valley appears**
- The bottom of the valley = the optimal model size for that budget
- Repeat for 9 different budgets

No training curves needed — just the final result of each fully trained model.

$$N_{\text{opt}} \propto C^{0.49} \quad D_{\text{opt}} \propto C^{0.51}$$



[Figure 3: the loss valley at each budget level]

Approach 3: Why We Need a Third Method

The limitation of Approaches 1 and 2: they find the optimal point empirically — by looking at actual training runs and picking the winner. This works well, but has a constraint: **They can only tell you the optimal point for compute budgets you actually tested.**

What if you want to predict the optimal for a budget you never ran?

You need a smooth mathematical function fitted to all the data — one that lets you predict the optimal N and D at any point on the compute axis, including scales far beyond your experiments.

That is exactly what Approach 3 builds.

- Uses **all losses** from Approaches 1 & 2 as input data
- Fits a single closed-form function $\hat{L}(N, D)$ to all 400+ runs
- Derives the optimal frontier **analytically** via calculus
- Can predict the optimal allocation at **any compute budget**

Approaches 1 & 2 find the optimal empirically. Approach 3 derives it analytically.

Approach 3: The Loss Function

$$\hat{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

Term	Intuition
E	The floor — entropy of natural text. No model ever beats this.
$+ A/N^\alpha$	Your model is finite. Even perfect training leaves a gap. Bigger N shrinks it.
$+ B/D^\beta$	Your model has not fully converged. More tokens shrink this gap.

Fit parameters (A, B, E, α, β) to all observed losses, then minimize $\hat{L}$ subject to $C \approx 6ND$ — the optimal frontier follows directly.

$$N_{\text{opt}} \propto C^{0.46} \quad D_{\text{opt}} \propto C^{0.54}$$

 [Figure 4: iso loss contours with the efficient frontier in blue. The blue line is the optimal path through the loss landscape at every compute level.]

All Three Approaches Converge

The result is robust across independent methods

Approach	a	b	10th–90th pct (a / b)
1. Min over training curves	0.50	0.50	(0.488, 0.502) / (0.501, 0.512)
2. IsoFLOP profiles	0.49	0.51	(0.462, 0.534) / (0.483, 0.529)
3. Parametric modelling	0.46	0.54	(0.454, 0.455) / (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27	—

All three cluster at $a \approx b \approx 0.50$. Kaplan's $a = 0.73$ is not a small discrepancy — it is a fundamental disagreement.

What This Means Concretely

For every doubling of model size → double the training tokens.

In other words: **parameters and data should always grow together.**

Model	Parameters	Tokens	Tokens needed (optimal)
GPT-3	175B	300B	~3.3T
Gopher	280B	300B	~5.4T
MT-NLG	530B	270B	~10T
Chinchilla	70B	1.4T	~1.4T ✓

Every model except Chinchilla was trained on a fraction of the data it actually needed.

 [Figure 1: GPT-3, Gopher, MT-NLG all sitting above the optimal frontier. Chinchilla sitting on it.]

ACT 3: The Payoff

Chinchilla — building and testing the compute-optimal model

The Prediction: What Should We Build?

We now have a rule: **parameters and tokens should scale equally with compute.**

So let's apply it to a real model.

Gopher was one of the largest models at the time:

- 280B parameters — 300B tokens — budget of 5.76×10^{23} FLOPs

Was that a good allocation? - We can check by plugging the budget into the optimal frontier.

Optimal allocation for the same budget:

	Gopher (actual)	Optimal (predicted)
Parameters	280B	~70B
Tokens	300B	~1.4T
Compute	5.76×10^{23}	5.76×10^{23}

The Test: Can We Verify This?

This is a prediction. The only way to verify it is to actually train the optimal model and see if it wins.

The authors decide to train exactly what the frontier recommends:

- Take **Gopher's compute budget** — don't add a single extra FLOP
- Build a model at the predicted optimal point: **70B parameters**
- Train it for the predicted optimal duration: **1.4 trillion tokens**
- Keep everything else identical — same architecture family, same data source

This model is **Chinchilla**.

Same compute. Same data source. Same architecture family.

Only the N/D split changes.

Any performance difference between Chinchilla and Gopher is **directly and exclusively attributable** to how the compute budget was allocated.

Chinchilla vs. Gopher: The Setup

	Gopher	Chinchilla
Parameters	280B	70B (4× smaller)
Training Tokens	300B	1.4T (4.7× more)
Compute Budget	5.76×10^{23} FLOPs	5.76×10^{23} FLOPs
Architecture	Same family	Same family
Training Data	MassiveText	MassiveText
N/D allocation	Over-parameterized	Compute-optimal ✓

 [Figure 1: the Gopher compute budget vertical line. Gopher sits above the frontier. Chinchilla sits directly on it.]

Does Chinchilla Actually Win?

Chinchilla (70B) was tested against models 2× to 4× its size.

Category	vs Gopher 280B	vs GPT-3 175B
Language modelling	✓ Wins all subsets	✓ Wins
MMLU (knowledge)	✓ +7.5pp	✓ Wins
Reasoning & comprehension	✓ +10.7pp	✓ Wins
Common sense	✓ Wins all	✓ Wins all
Closed-book QA	✓ New SOTA	✓ Wins
Truthfulness	✓ +14.1pp	—

A 70B model beats models 2× to 4× its size across every category tested.

Knowledge: Language Modelling and MMLU

Perplexity — how surprised is the model by text? Lower = better.

	Chinchilla 70B	Gopher 280B
Wikitext103	7.16	7.75

MMLU (Measuring Massive Multitask Language Understanding) — 57 academic subjects, exam-style. Like a university entrance exam across every discipline at once.

Model	Score	Params
Chinchilla	67.5%	70B
Gopher	60.0%	280B
GPT-3	~43.9%	175B

Reasoning, Comprehension and Common Sense

BIG-bench — 200+ tasks requiring multi-step reasoning, not memorization:

Chinchilla **65.1%** vs Gopher **54.4%** — gap of **+10.7pp**

RACE — real exam questions from high school and middle school:

	Chinchilla 70B	Gopher 280B	GPT-3 175B
RACE-h	82.3%	71.6%	46.8%
RACE-m	86.8%	75.1%	58.1%

Common sense — everyday reasoning humans do automatically:

	Chinchilla 70B	Gopher 280B	GPT-3 175B
Winogrande	74.9%	70.1%	70.2%
BoolQ	83.7%	79.3%	60.5%
HellaSwag	80.8%	79.2%	78.9%

Truthfulness and Factual Recall

Natural Questions (closed-book) — factual questions answered from memory alone, no retrieval:

	Chinchilla 70B	Gopher 280B	GPT-3 175B
5-shot	31.5%	21.0%	14.1%
64-shot	35.5%	28.0%	—

TruthfulQA — does the model repeat false beliefs common among humans?

Higher = more truthful.

	Chinchilla 70B	Gopher 280B	Δ
0-shot	43.6%	29.5%	+14.1pp
10-shot	66.7%	43.7%	+23pp

A 14-point truthfulness improvement from pre-training alone. Compute-optimal training makes models more truthful — not just more capable.

The Practical Argument

The benefits compound after training ends

Training

- Identical FLOPs to Gopher
- No additional cost
- Better performance from N/D allocation alone

Deployment

- 4× smaller memory footprint
- Fits on smaller hardware
- Broader accessibility

Inference

- Cheaper per call — fewer parameters
- Savings compound at scale

Fine-tuning

- Cheaper to adapt for downstream tasks
- Directly relevant for research labs

Broader Implications

The Chinchilla finding is not just about one model — it changes how we think about scaling.

More data is not automatically better.

The paper is explicit: larger datasets only help when the data is high quality. Scraping more text from the web brings more toxic content, biases, and private information — not just more knowledge. Dataset quality is now as important as dataset size.

A better trained model is a safer model.

Chinchilla shows less toxicity than Gopher despite training on the same data source. No special safety training — just better pre-training. Compute-optimal training appears to benefit alignment as a side effect.

The method is reproducible.

The paper describes the three approaches as "*easy to reproduce in new settings*" — an open invitation to apply this framework to new domains and scales.

■ The bottleneck has shifted. It is no longer compute — it is data quality.

Key Takeaway

✗ Old Rule (Kaplan)

- Parameters scale at $a = 0.73$
- Data fixed at ~300B tokens
- Bigger = better
- GPT-3, Gopher, MT-NLG all followed this

	Gopher	Chinchilla
Parameters	280B	70B
Tokens	300B	1.4T
Compute	same	same
Performance	baseline	 wins everywhere

New Rule (Chinchilla)

- Scale N and D equally: $a \approx b \approx 0.50$
- More tokens = more value
- Smaller + longer wins
- Confirmed by three independent methods

For every doubling of model size → double the training tokens.